Seat Booking (BookMyShow / Flight)

- **Problem:** Build a seat selection screen for a movie show (or a flight). Show a grid of seats grouped by price tier, mark seats that are sold or locked by other users, let the user pick up to N seats, show the total, then hold the seats for a few minutes while the user pays. If someone else grabs a seat first, or the hold runs out, handle it cleanly.
- **Tech stack:** React (Vite, JavaScript, Tailwind CSS v4)
- **Time limit:** 60 minutes
- **Often asked at:** BookMyShow, MakeMyTrip, Swiggy
- **Why it matters:** Grid state, locked or reserved seats

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and its expected result. This is your checklist for building and testing.
3. **Design the architecture.** Name the components and their one-line jobs. Start small.
4. **Design the state.** What is the single source of truth? What can be derived?
5. **Design data structures and contracts.** Shape of the data, API inputs and outputs, error codes.
6. **Trace one data flow end to end.** Proves the design works before you type.
7. **Find the hard parts.** Solve them on paper first.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.** Loading, error, empty, conflicts, expiry, double clicks.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state design	7 min
Hard parts on paper (409 conflict, hold timer, booking state machine)	5 min
Build: mock API + render the grid from data	12 min
Build: select / deselect, max seats, total	8 min
Build: hold + countdown + pay + cancel	13 min
Build: conflict (409) and expiry (410) handling	5 min
Edge cases, accessibility, wrap-up	5 min

Tip: think out loud

Say what you are deciding and why: "The frontend can't guarantee a seat is free, because two people can click the same seat at the same moment. So the server owns seat status and holds. My UI shows what the server said last, and treats a 409 as a normal path, not a crash." The interviewer grades your reasoning as much as the code.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Load the seat layout for one show: rows, seats per row, aisles, price tier per row.
- Show each seat as **available**, **selected**, **locked** (held by someone else) or **sold**.
- Click an available seat to select it, click again to deselect it. Sold and locked seats can't be clicked.
- Limit selection to a maximum (6 seats).
- Show the selected seat numbers and the running total.
- **Proceed** holds (locks) the selected seats on the server for a fixed time (2 minutes in the demo) and shows a countdown.
- **Pay** confirms the booking. **Change seats** releases the hold.
- If a seat was taken by someone else before the hold, tell the user which one, mark it sold and drop it from the selection.
- If the hold expires, release the seats and go back to selection.
- Show a confirmation with the booking id.

Non-functional requirements

Area	What it means here
Performance	~120 seats today, up to ~500 in a big hall or a wide-body plane. A click must re-render only the seats that changed, not the whole grid. The countdown ticks every second without re-rendering the map.
Correctness under concurrency	The server is the source of truth. The UI must survive "someone else booked it first" (409) and "your hold expired" (410).
Error handling	Load failure shows retry. Hold/pay failures show a clear message and leave the user in a sane state. Double clicks never double book.
Accessibility	Every seat is a real `<button>` with an `aria-label` ("Seat F6, ₹250, available") and `aria-pressed`. Status is not shown by colour alone (label + legend). Messages go to an `aria-live` region.
Reusability	The layout comes from data (rows, tiers, aisles). The same code renders a cinema or a flight cabin.
Extensibility	Pure reducer for the booking flow, so new rules (no single-seat gaps, seat categories, multiple shows) slot in without touching components.

Clarifying questions to ask the interviewer

1. Is there a real API, or should I mock it? What does the layout payload look like?
2. Do I need the "lock / hold" step with a timer, or just select and book?
3. Is there a maximum number of seats per booking? Is the seat count chosen up front (BookMyShow asks "How many seats?") or free?
4. Do I need live updates when other users book seats (WebSocket / polling), or is a 409 on hold enough?
5. Any layout gaps: aisles, missing seats, wheelchair spots, different tiers per row?
6. Any business rules like "don't leave a single empty seat between two booked seats"?

Assumptions we will build with

- A mock API in the same project, with fake latency, that behaves like a real server: it owns seat status and holds, and returns 409 / 410 errors.
- One show, 10 rows x 12 seats, two aisles, three tiers (Recliner ₹450, Prime ₹250, Classic ₹180).
- Free selection, max 6 seats.
- Hold lasts 2 minutes (short so it can be demoed; production is usually 5 to 10).
- No live updates; conflicts are found when holding. Live updates are a "with more time" item.
- No real payment: **Pay** calls `confirmBooking(holdId)`.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page opens	"Loading seats…", then the grid grouped by tier, with legend and summary bar
Layout request fails	Error message + **Retry** button
Click an available seat	Seat turns green (selected), summary shows it and the total
Click a selected seat	Seat is deselected, total goes down
Click a sold or locked seat	Nothing (button is disabled)
Pick a 7th seat	Selection stays at 6, message "You can pick up to 6 seats."
Proceed with 0 seats	Button is disabled
Proceed with seats	Button shows "Holding seats…", then the countdown and **Pay** / **Change seats** appear. The grid is locked.
Proceed, but one seat was taken meanwhile (409)	Message names the seat, that seat becomes sold and is removed from the selection, the rest stay selected
Change seats	Hold released on the server, back to selection with the same seats selected
Countdown reaches 00:00	Back to selection, message "Your seats were released…"
Pay	Button shows "Paying…", then the confirmation with booking id, seats and total. Booked seats become sold.
Pay after the server already expired the hold (410)	Message "Your hold expired…", back to selection

Action / input	Expected behavior / output
Pay double click	Second click does nothing (button disabled while paying)
Book more seats	Selection clears, grid unlocks
Keyboard: Tab / Enter / Space	Moves through available seats and toggles them; focus ring is visible

This table is the implementation and test checklist.

---

3. Design the architecture

Module	Responsibility
`api/seatApi.js`	Mock server: `fetchLayout`, `holdSeats`, `confirmBooking`, `releaseHold`. Owns the "database" and the 409 / 410 rules.
`utils/booking.js`	Pure logic: `normalizeLayout`, `bookingReducer` (the booking state machine), selectors `getSelectedSeats`, `getTotal`.
`utils/format.js`	`formatPrice` (₹, en-IN), `formatTimeLeft` (mm:ss).
`hooks/useSeatBooking.js`	Connects reducer + API: load, hold, pay, cancel, hold expiry timer.
`components/SeatMap.jsx`	Renders rows, tier headings, aisles and the screen.
`components/Seat.jsx`	One memoized seat button.
`components/Legend.jsx`	Colour key.
`components/HoldTimer.jsx`	Countdown that ticks on its own.
`components/BookingSummary.jsx`	Selected seats, total, Proceed / Pay / Change seats / confirmation.
`App.jsx`	Loading / error screens and layout.

```
App
 ├─ useSeatBooking(showId) ──► bookingReducer (pure)
 │        │                        ▲
 │        └──► seatApi (mock server: fetchLayout / holdSeats / confirmBooking / releaseHold)
 ├─ Legend
 ├─ SeatMap
 │    └─ Seat x 120 (memo)
 ├─ notice (aria-live)
 └─ BookingSummary
      └─ HoldTimer (ticks every 1 s, only this re-renders)
```

Start simple: first version is `App` with the grid and a `useState` selection. Split out the reducer the moment a second "mode" (holding / paying) appears, because then many `useState`s would have to change together.

---

4. Design the state / data model

State or field	Purpose
`status`	`'loading' | 'error' | 'ready'` for the layout request
`error`	Message for the load error screen
`layout`	Static data: show info, tiers + prices, `maxSeats`, `aisleAfter`, rows as `{ label, tier, seatIds }`
`seats`	`{ [seatId]: { id, number, row, tier, status } }` with status `'available' | 'held' | 'booked'` as the server last told us
`selected`	Array of seat ids in pick order
`phase`	`'select' | 'holding' | 'held' | 'paying' | 'booked'` the booking state machine
`hold`	`{ id, expiresAt }` while seats are held
`bookingId`	Confirmation id after payment
`notice`	One message for the user (max seats, 409, 410, expiry)

```
export const initialState = {
  status: 'loading',
  error: null,
  layout: null,
  seats: {},
  selected: [],
  phase: 'select',
  hold: null,
  bookingId: null,
  notice: null,
};
```

Senior notes

- **Derive, don't store:** total price, the list of selected seat objects, "is this seat selected" (`Set` built from `selected`), "is the grid locked" (`phase !== 'select'`), and the seconds left on the timer (`expiresAt - now`). Storing any of these means they can go out of sync.
- **"Selected" is not a seat status.** Seat status is server truth (`available / held / booked`). Selection is the user's local choice. Keeping them separate means a server refresh can never wipe or fake a selection.
- **One `phase` instead of booleans.** `isHolding` , `isHeld` , `isPaying` , `isBooked` as four booleans allow impossible combos (`isPaying && !isHeld`). One enum makes the impossible states impossible.
- **Ownership:** all booking state lives in `useSeatBooking` (one reducer). `HoldTimer` owns only `now` (its tick). `Seat` owns nothing.

---

5. Design the data structures and contracts

Layout payload (what the server sends)

```
{
  "show": { "id": "show-42", "title": "Interstellar (Re-release)", "venue": "PVR Phoenix, Audi 3",
"time": "Sun, 8 Nov, 7:30 PM" },
  "tiers": {
    "recliner": { "label": "Recliner", "price": 450 },
    "prime":    { "label": "Prime",    "price": 250 },
    "classic":  { "label": "Classic",  "price": 180 }
  },
  "maxSeats": 6,
  "aisleAfter": [3, 9],
  "rows": [
    { "label": "J", "tier": "recliner", "seats": [ { "id": "J1", "number": 1, "status": "available" },
"..." ] }
  ]
}
```

Why this shape:

- **Stable seat ids** (`"J1"`) are the real identity. They are the React `key` , the thing we send to the server, and what the user reads. Never use the array index.
- **Price lives on the tier, not the seat.** One place to change it; the seat stores only `tier` .
- **Aisles as data** (`aisleAfter`), so a flight cabin (`3-3` or `2-4-2`) is just different data.

Normalized client shape

We flatten seats into a map by id so updating one seat (after a 409) is one object spread, and lookups are O(1):

```
// rows: [{ label: 'J', tier: 'recliner', seatIds: ['J1', 'J2', ...] }]
// seats: { J1: { id: 'J1', number: 1, row: 'J', tier: 'recliner', status: 'available' }, ... }
```

API contracts (mocked)

Function	Input	Success	Errors
`fetchLayout(showId)`	show id	layout payload above	network error -> load error screen
`holdSeats(showId, seatIds)`	1 to 6 seat ids	`{ holdId: 'hold_1a2b3c4d', expiresInMs: 120000 }`	`400` bad count; `409 { taken: ['F6'] }` some seats not available
`confirmBooking(holdId)`	hold id	`{ bookingId: 'BMS482913', seatIds: ['F5', 'F6'] }`	`410` hold expired or unknown

Function	Input	Success	Errors
`releaseHold(holdId)`	hold id	nothing	ignored (server TTL frees seats anyway)

Errors carry `status` (and `taken` for 409) so the hook can branch on them:

```
function apiError(status, message, extra = {}) {
  return Object.assign(new Error(message), { status, ...extra });
}
```

---

6. Trace the data flow

Interaction: user selected F5 and F6 and clicks "Proceed", but F6 was booked by someone else 2 seconds ago.

```
Click "Proceed ₹500"
   │
   ▼
hold()  ──► dispatch({ type: 'holdStart' })
   │            phase: 'select' -> 'holding'
   │            grid locked, button shows "Holding seats…"
   ▼
await holdSeats('show-42', ['F5', 'F6'])
   │        server checks ALL seats first: F6 is 'booked'
   │        nothing is held (all-or-nothing)
   ▼
throw { status: 409, taken: ['F6'], message: 'Sorry, F6 just got booked…' }
   │
   ▼
dispatch({ type: 'holdFail', taken: ['F6'], message })
   │        seats.F6.status = 'booked'
   │        selected = ['F5']
   │        phase = 'select', notice = message
   ▼
Re-render
   • F6 turns grey (sold) and is disabled — only F6 and F5's props changed, so only they re-render
   • aria-live region reads the message
   • summary: "1 seat: F5", "Proceed ₹250"
```

The happy path continues the same way: `holdOk` -> `phase: 'held'`, `hold: { id, expiresAt }` -> the expiry `setTimeout` is armed -> `HoldTimer` mounts and ticks -> **Pay** -> `payStart` -> `confirmBooking` -> `payOk` -> seats marked sold, `phase: 'booked'`.

---

7. Important technical challenges

Challenge 1: Two users click the same seat (the 409 conflict)

Problem: Alice and Bob both see F6 as available. Both click it. If the UI decides "it's free, book it", one of them gets a ticket that doesn't exist. The client can never know about the other user's click.

Solution: The server owns seat status. Holding is **all-or-nothing**: check every seat first, and only if all are free mark them held. In a real backend this is a DB transaction or `SET NX` per seat in Redis. The client treats 409 as a normal outcome: mark the taken seats sold, keep the rest of the selection, explain.

```
// server (mock)
const taken = seatIds.filter((id) => seatStatus.get(id) !== 'available');
if (taken.length) throw apiError(409, `Sorry, ${taken.join(', ')} just got booked…`, { taken });
seatIds.forEach((id) => seatStatus.set(id, 'held'));

// client reducer
case 'holdFail': {
  const taken = action.taken ?? [];
  return {
    ...state,
    phase: 'select',
    seats: markBooked(state.seats, taken),
    selected: state.selected.filter((id) => !taken.includes(id)),
    notice: action.message,
  };
}
```

Challenge 2: The hold timer (drift, clock skew, stale timers, expiring mid-payment)

Problem: Four traps hide in "show a 2 minute countdown":

1. Counting ticks (`seconds--` every 1 s) drifts: background tabs throttle `setInterval` to once a minute, so the UI would show 1:30 left when the server already released the seats.
2. Using the server's `expiresAt` timestamp breaks when the user's clock is 3 minutes wrong.
3. User cancels and holds again: the first hold's timer fires and expires the **new** hold.
4. Timer hits 0 while the payment request is in flight. If the client "expires" locally, it throws away a payment the server may have accepted.

Solution:

1. Store a deadline, derive time left: `left = expiresAt - Date.now()`.
2. Server sends a **duration** (`expiresInMs`); the client computes its own deadline from its own clock on receipt.
3. The `expire` action carries the `holdId`; the reducer ignores it if it doesn't match.
4. Never expire locally while `phase === 'paying'`; the pay response (success or 410) decides.

```
// hook: duration -> local deadline
dispatch({ type: 'holdOk', hold: { id: res.holdId, expiresAt: Date.now() + res.expiresInMs } });

// hook: one timeout per hold, cleaned up when the hold changes
useEffect(() => {
  if (!state.hold) return;
  const { id, expiresAt } = state.hold;
  const timer = setTimeout(() => dispatch({ type: 'expire', holdId: id }), expiresAt - Date.now());
  return () => clearTimeout(timer);
}, [state.hold]);

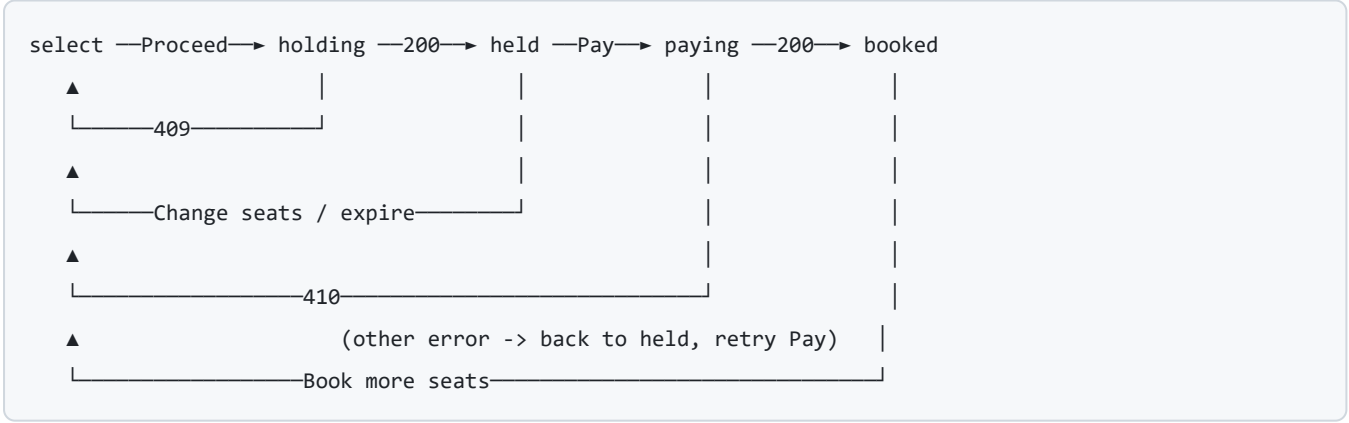
// reducer
case 'expire':
  if (state.hold?.id !== action.holdId || state.phase !== 'held') return state;
  return { ...state, phase: 'select', hold: null, notice: 'Your seats were released because the timer
ran out.' };
```

The client timer is only a mirror for the user. The server's TTL is the real rule, so even if the tab is closed, the seats come back.

Challenge 3: The booking flow is a state machine, not a pile of booleans

Problem: With `useState` booleans you get bugs like: user clicks a seat while "Holding…" (selection changes after we sent it), double clicks **Pay** (two bookings), clicks **Change seats** while paying.

Solution: One `phase` and a reducer that ignores actions not allowed in the current phase. The UI also disables the buttons, but the reducer is the real guard.




```
case 'toggle': {
  if (state.phase !== 'select') return state;          // grid locked
  ...
}
case 'payStart':
  return state.phase === 'held' ? { ...state, phase: 'paying', notice: null } : state; // no double pay
```

Challenge 4: Re-rendering 120 to 500 seats on every click

Problem: Clicking one seat creates a new `selected` array. Without care, all 120 `Seat`s re-render, and the countdown ticking every second would re-render the whole map 120 times a minute.

Solution:

- `Seat` is `memo`ized and gets only primitive / stable props: `seat` object (same reference unless that seat changed), `price`, `isSelected` boolean, `locked` boolean, and `onToggle` from `useCallback` over `dispatch` (stable forever).
- `selectedSet` is built once per selection change with `useMemo`, so `isSelected` is O(1).
- The countdown lives in its own `HoldTimer` component, so its 1 s tick re-renders one `<p>`.

```
const toggleSeat = useCallback((id) => dispatch({ type: 'toggle', id }), []);
const selectedSet = useMemo(() => new Set(selected), [selected]);
<Seat seat={seats[id]} isSelected={selectedSet.has(id)} locked={locked} onToggle={onToggle} price=
{tier.price} />
export default memo(Seat);
```

---

8. Implement in the correct order

#	Step	Working result
1	Mock API: `fetchLayout` with fake latency and pre-booked seats	`console.log` shows the payload
2	`App` loads the layout, shows loading / error / Retry	Screen says "Loading seats…" then data
3	`normalizeLayout` + `SeatMap` + `Seat`: rows, tier headings, aisles, screen	Static grid with sold seats grey
4	Selection with `useState` array: toggle, disabled sold seats	Seats turn green on click
5	Max 6 seats + `notice`; summary with seat list and total (`formatPrice`)	Total updates live
6	Move to `bookingReducer` + `useSeatBooking`; add `phase`	Same behaviour, now testable
7	`holdSeats` in the mock + Proceed -> `holding` -> `held`; lock the grid	Proceed shows Pay / Change seats
8	`HoldTimer` + expiry `setTimeout` keyed by `holdId`	Countdown runs, expiry returns to select
9	`confirmBooking` + Pay -> `paying` -> `booked` + confirmation	Full happy path works
10	`releaseHold` + Change seats	Cancel frees seats
11	409 (`taken`) and 410 (expired) handling	Conflict and expiry paths work
12	Accessibility (labels, `aria-pressed`, live region, focus ring), `memo`, Legend	Polish

---

9. Cover all states and edge cases

State / edge case	What the user sees / what happens
Initial / loading	"Loading seats…" with `role="status"`
Load error	Message + Retry; Retry ignores responses from older requests (`loadId` ref)
Empty selection	"Pick up to 6 seats", Proceed disabled
Max seats reached	7th click ignored, notice shown
Seat sold / locked by others	Grey / amber, disabled, label says "sold" / "locked by another user"
Holding (request in flight)	"Holding seats…", grid locked, Proceed disabled
Hold conflict (409)	Taken seats become sold and leave the selection; message lists them
All selected seats taken (409)	Selection becomes empty, Proceed disabled again
Held	Countdown, Pay, Change seats, grid locked
Countdown under 30 s	Timer turns red
Hold expired locally	Back to select, selection kept, message
Hold expired on server before Pay (410)	Back to select, selection kept so the user can try again
Pay fails with other error (network)	Stay in held, message, user can retry Pay while the timer runs
Timer hits 0 during Pay	Ignored locally; server response decides
Double click Proceed / Pay	Second click blocked by phase guard + disabled button
Old timer from a cancelled hold	Ignored (holdId mismatch)
Booked	Confirmation with id, seats, total; seats turn sold; "Book more seats"
User closes the tab while held	Server TTL releases the seats
Narrow screen	Grid scrolls horizontally inside its box, page doesn't
Background tab	Countdown is computed from the deadline, so it's correct when the tab comes back
StrictMode double effects	Second load wins via `loadId`; timers are cleaned up

---

10. Final structure

```
_72_seat_booking/
├─ index.html
├─ package.json
├─ vite.config.js
└─ src/
   ├─ main.jsx
   ├─ index.css
```

```
   ├─ App.jsx
   ├─ api/
   │  └─ seatApi.js          # mock server: layout, hold (409), confirm (410), release
   ├─ utils/
   │  ├─ booking.js          # normalizeLayout, bookingReducer, selectors
   │  └─ format.js           # formatPrice, formatTimeLeft
   ├─ hooks/
   │  └─ useSeatBooking.js   # reducer + API calls + expiry timer
   └─ components/
      ├─ SeatMap.jsx
      ├─ Seat.jsx
      ├─ Legend.jsx
      ├─ HoldTimer.jsx
      └─ BookingSummary.jsx
```

Setup: `npm create vite@latest` (React, JavaScript), then `npm i tailwindcss @tailwindcss/vite`.

---

11. Full working code

vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

src/index.css

```
@import "tailwindcss";
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

src/api/seatApi.js

```
// Mock backend. The Maps below play the role of the server's database,
// so every rule that matters (409 on taken seats, hold expiry) is enforced HERE, not in the UI.

const TIERS = {
  recliner: { label: 'Recliner', price: 450 },
  prime: { label: 'Prime', price: 250 },
  classic: { label: 'Classic', price: 180 },
};
// Display order: back of the hall first, screen is drawn at the bottom
const ROWS = [
  ['J', 'recliner'], ['I', 'recliner'],
  ['H', 'prime'], ['G', 'prime'], ['F', 'prime'],
  ['E', 'classic'], ['D', 'classic'], ['C', 'classic'], ['B', 'classic'], ['A', 'classic'],
];
const COLS = 12;
const MAX_SEATS = 6;
const HOLD_MS = 2 * 60 * 1000; // short so you can watch a hold expire
const PRE_BOOKED = ['J5', 'J6', 'H3', 'H4', 'G10', 'E1', 'E2', 'C7', 'C8', 'A11', 'A12'];
// Another user books these 8 s after you load the page: pick one of them to see a 409
const GRABBED_LATER = ['F6', 'F7', 'G6', 'G7'];

const seatStatus = new Map(); // seatId -> 'available' | 'held' | 'booked'
const holds = new Map(); // holdId -> { seatIds, expiresAt }

for (const [row] of ROWS) {
  for (let n = 1; n <= COLS; n++) seatStatus.set(`${row}${n}`, 'available');
}
PRE_BOOKED.forEach((id) => seatStatus.set(id, 'booked'));

const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

function apiError(status, message, extra = {}) {
  return Object.assign(new Error(message), { status, ...extra });
}

// A real server does this with a TTL (Redis EXPIRE / a cron); here we sweep on every call
function sweepExpiredHolds() {
  const now = Date.now();
  for (const [holdId, hold] of holds) {
    if (hold.expiresAt > now) continue;
    hold.seatIds.forEach((id) => seatStatus.set(id, 'available'));
    holds.delete(holdId);
  }
}
```

```
export async function fetchLayout(showId) {
  await wait(500);
  sweepExpiredHolds();
  setTimeout(() => {
    GRABBED_LATER.forEach((id) => {
      if (seatStatus.get(id) === 'available') seatStatus.set(id, 'booked');
    });
  }, 8000);

  return {
    show: { id: showId, title: 'Interstellar (Re-release)', venue: 'PVR Phoenix, Audi 3', time: 'Sun, 8
Nov, 7:30 PM' },
    tiers: TIERS,
    maxSeats: MAX_SEATS,
    aisleAfter: [3, 9],
    rows: ROWS.map(([label, tier]) => ({
      label,
      tier,
      seats: Array.from({ length: COLS }, (_, i) => {
        const id = `${label}${i + 1}`;
        return { id, number: i + 1, status: seatStatus.get(id) };
      }),
    })),
  };
}

// All-or-nothing: either every seat is held for you, or none is
export async function holdSeats(showId, seatIds) {
  await wait(400);
  sweepExpiredHolds();
  if (seatIds.length === 0 || seatIds.length > MAX_SEATS) {
    throw apiError(400, `Pick 1 to ${MAX_SEATS} seats.`);
  }
  const taken = seatIds.filter((id) => seatStatus.get(id) !== 'available');
  if (taken.length) {
    throw apiError(409, `Sorry, ${taken.join(', ')} just got booked by someone else. Please pick other
seats.`, { taken });
  }
  const holdId = `hold_${crypto.randomUUID().slice(0, 8)}`;
  seatIds.forEach((id) => seatStatus.set(id, 'held'));
  holds.set(holdId, { seatIds, expiresAt: Date.now() + HOLD_MS });
  return { holdId, expiresInMs: HOLD_MS };
}

export async function confirmBooking(holdId) {
  await wait(700);
  sweepExpiredHolds();
```

```
  const hold = holds.get(holdId);
  if (!hold) throw apiError(410, 'Your hold expired before payment. Please pick your seats again.');
  hold.seatIds.forEach((id) => seatStatus.set(id, 'booked'));
  holds.delete(holdId);
  return { bookingId: `BMS${String(Date.now()).slice(-6)}`, seatIds: hold.seatIds };
}

export async function releaseHold(holdId) {
  await wait(200);
  const hold = holds.get(holdId);
  if (!hold) return;
  hold.seatIds.forEach((id) => seatStatus.set(id, 'available'));
  holds.delete(holdId);
}
```

src/utils/booking.js

```
// Pure booking logic: no React, no fetch, so it can be unit tested with plain asserts.

export const initialState = {
  status: 'loading', // 'loading' | 'error' | 'ready'
  error: null,
  layout: null, // { show, tiers, maxSeats, aisleAfter, rows: [{ label, tier, seatIds }] }
  seats: {}, // seatId -> { id, number, row, tier, status }
  selected: [], // seat ids in the order the user picked them
  phase: 'select', // 'select' | 'holding' | 'held' | 'paying' | 'booked'
  hold: null, // { id, expiresAt }
  bookingId: null,
  notice: null, // message for the user (409, 410, max seats)
};

// API sends rows with nested seats; we store seats by id so one seat update is O(1)
export function normalizeLayout(data) {
  const seats = {};
  const rows = data.rows.map((row) => ({
    label: row.label,
    tier: row.tier,
    seatIds: row.seats.map((seat) => {
      seats[seat.id] = { ...seat, row: row.label, tier: row.tier };
      return seat.id;
    }),
  }));
  const { show, tiers, maxSeats, aisleAfter } = data;
  return { layout: { show, tiers, maxSeats, aisleAfter, rows }, seats };
}

function markBooked(seats, ids) {
```

```
  const next = { ...seats };
  ids.forEach((id) => {
    if (next[id]) next[id] = { ...next[id], status: 'booked' };
  });
  return next;
}

export function bookingReducer(state, action) {
  switch (action.type) {
    case 'loadStart':
      return initialState;
    case 'loadOk':
      return { ...initialState, status: 'ready', ...normalizeLayout(action.data) };
    case 'loadFail':
      return { ...state, status: 'error', error: action.message };

    case 'toggle': {
      if (state.phase !== 'select') return state;
      const seat = state.seats[action.id];
      if (!seat || seat.status !== 'available') return state;
      if (state.selected.includes(action.id)) {
        return { ...state, selected: state.selected.filter((id) => id !== action.id), notice: null };
      }
      const max = state.layout.maxSeats;
      if (state.selected.length >= max) return { ...state, notice: `You can pick up to ${max} seats.`
};
      return { ...state, selected: [...state.selected, action.id], notice: null };
    }

    case 'holdStart':
      if (state.phase !== 'select' || state.selected.length === 0) return state;
      return { ...state, phase: 'holding', notice: null };
    case 'holdOk':
      return { ...state, phase: 'held', hold: action.hold };
    case 'holdFail': {
      const taken = action.taken ?? [];
      return {
        ...state,
        phase: 'select',
        seats: markBooked(state.seats, taken),
        selected: state.selected.filter((id) => !taken.includes(id)),
        notice: action.message,
      };
    }

    case 'payStart':
      return state.phase === 'held' ? { ...state, phase: 'paying', notice: null } : state;
```

```
    case 'payOk':
      return {
        ...state,
        phase: 'booked',
        seats: markBooked(state.seats, state.selected),
        hold: null,
        bookingId: action.bookingId,
      };
    case 'payFail':
      // 410 = hold is gone on the server, start over; anything else = hold still valid, let them retry
      return action.expired
        ? { ...state, phase: 'select', hold: null, notice: action.message }
        : { ...state, phase: 'held', notice: action.message };

    case 'expire':
      // Ignore a timer from an older hold, and never expire locally while payment is in flight
      if (state.hold?.id !== action.holdId || state.phase !== 'held') return state;
      return { ...state, phase: 'select', hold: null, notice: 'Your seats were released because the
timer ran out.' };
    case 'cancel':
      return state.phase === 'held' ? { ...state, phase: 'select', hold: null, notice: null } : state;
    case 'newBooking':
      return { ...state, phase: 'select', selected: [], bookingId: null, notice: null };

    default:
      return state;
  }
}

export function getSelectedSeats(state) {
  return state.selected.map((id) => {
    const seat = state.seats[id];
    return { ...seat, price: state.layout.tiers[seat.tier].price };
  });
}

export function getTotal(state) {
  return getSelectedSeats(state).reduce((sum, seat) => sum + seat.price, 0);
}
```

src/utils/format.js

```
const inr = new Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR', maximumFractionDigits:
0 });

export const formatPrice = (amount) => inr.format(amount);
```

```
// 90500 -> "01:31" (ceil so the timer shows 00:00 only when time is really up)
export function formatTimeLeft(ms) {
  const total = Math.max(0, Math.ceil(ms / 1000));
  const mm = String(Math.floor(total / 60)).padStart(2, '0');
  const ss = String(total % 60).padStart(2, '0');
  return `${mm}:${ss}`;
}
```

src/hooks/useSeatBooking.js

```
import { useCallback, useEffect, useReducer, useRef } from 'react';
import { confirmBooking, fetchLayout, holdSeats, releaseHold } from '../api/seatApi';
import { bookingReducer, initialState } from '../utils/booking';

export function useSeatBooking(showId) {
  const [state, dispatch] = useReducer(bookingReducer, initialState);
  const loadId = useRef(0);

  const load = useCallback(async () => {
    const id = ++loadId.current; // only the latest request may write state
    dispatch({ type: 'loadStart' });
    try {
      const data = await fetchLayout(showId);
      if (id === loadId.current) dispatch({ type: 'loadOk', data });
    } catch (err) {
      if (id === loadId.current) dispatch({ type: 'loadFail', message: err.message });
    }
  }, [showId]);

  useEffect(() => {
    load();
  }, [load]);

  // Local timer only mirrors the server TTL; the server releases the seats either way
  useEffect(() => {
    if (!state.hold) return;
    const { id, expiresAt } = state.hold;
    const timer = setTimeout(() => dispatch({ type: 'expire', holdId: id }), expiresAt - Date.now());
    return () => clearTimeout(timer);
  }, [state.hold]);

  // dispatch is stable, so this never changes and memoized seats never re-render because of it
  const toggleSeat = useCallback((id) => dispatch({ type: 'toggle', id }), []);

  async function hold() {
    dispatch({ type: 'holdStart' });
    try {
```

```
      const res = await holdSeats(showId, state.selected);
      // Use the server's duration, not its clock, so client clock skew can't shorten the hold
      dispatch({ type: 'holdOk', hold: { id: res.holdId, expiresAt: Date.now() + res.expiresInMs } });
    } catch (err) {
      dispatch({ type: 'holdFail', taken: err.taken, message: err.message });
    }
  }

  async function pay() {
    dispatch({ type: 'payStart' });
    try {
      const res = await confirmBooking(state.hold.id);
      dispatch({ type: 'payOk', bookingId: res.bookingId });
    } catch (err) {
      dispatch({ type: 'payFail', expired: err.status === 410, message: err.message });
    }
  }

  function cancel() {
    releaseHold(state.hold.id); // fire and forget: the TTL frees the seats if this call fails
    dispatch({ type: 'cancel' });
  }

  return {
    state,
    load,
    toggleSeat,
    hold,
    pay,
    cancel,
    newBooking: () => dispatch({ type: 'newBooking' }),
  };
}
```

src/components/Seat.jsx

```
import { memo } from 'react';

const LOOK = {
  available: 'border-emerald-600 text-emerald-700 hover:bg-emerald-50',
  selected: 'border-emerald-600 bg-emerald-600 text-white',
  held: 'border-amber-200 bg-amber-100 text-amber-400',
  booked: 'border-gray-200 bg-gray-200 text-gray-400',
};
const SPOKEN = { available: 'available', selected: 'selected', held: 'locked by another user', booked:
'sold' };
```

```
function Seat({ seat, price, isSelected, locked, onToggle }) {
  const look = isSelected ? 'selected' : seat.status;
  const unavailable = seat.status !== 'available';

  return (
    <button
      type="button"
      onClick={() => onToggle(seat.id)}
      disabled={unavailable || locked}
      aria-pressed={isSelected}
      aria-label={`Seat ${seat.id}, ₹${price}, ${SPOKEN[look]}`}
      title={`${seat.id} · ₹${price}`}
      className={`h-7 w-7 rounded-t-md border text-[11px] transition-colors focus-visible:outline-2
focus-visible:outline-offset-1 focus-visible:outline-emerald-700 disabled:cursor-not-allowed
${LOOK[look]}`}
    >
      {seat.number}
    </button>
  );
}

// 120 seats: only the one or two whose props changed re-render on a click
export default memo(Seat);
```

src/components/SeatMap.jsx

```
import { Fragment, useMemo } from 'react';
import Seat from './Seat';
import { formatPrice } from '../utils/format';

export default function SeatMap({ layout, seats, selected, locked, onToggle }) {
  const selectedSet = useMemo(() => new Set(selected), [selected]);

  return (
    <div className="overflow-x-auto pb-2">
      <div className="mx-auto w-max">
        {layout.rows.map((row, i) => {
          const tier = layout.tiers[row.tier];
          const firstOfTier = i === 0 || layout.rows[i - 1].tier !== row.tier;
          return (
            <Fragment key={row.label}>
              {firstOfTier && (
                <h3 className="mt-4 mb-1 border-b border-gray-200 pb-1 text-xs font-medium tracking-
wide text-gray-500 uppercase">
                  {tier.label} · {formatPrice(tier.price)}
                </h3>
              )}
```

```
              <div role="group" aria-label={`Row ${row.label}`} className="flex items-center gap-1.5
py-0.5">
                <span className="w-5 text-xs text-gray-500" aria-hidden="true">{row.label}</span>
                {row.seatIds.map((id, col) => (
                  <Fragment key={id}>
                    <Seat
                      seat={seats[id]}
                      price={tier.price}
                      isSelected={selectedSet.has(id)}
                      locked={locked}
                      onToggle={onToggle}
                    />
                    {layout.aisleAfter.includes(col + 1) && <span className="w-4" aria-hidden="true"
/>}
                  </Fragment>
                ))}
              </div>
            </Fragment>
          );
        })}

        <div className="mx-auto mt-8 h-1.5 w-3/4 rounded-full bg-sky-300" aria-hidden="true" />
        <p className="mt-1 text-center text-xs text-gray-500">All eyes this way please</p>
      </div>
    </div>
  );
}
```

src/components/Legend.jsx

```
const ITEMS = [
  ['Available', 'border-emerald-600'],
  ['Selected', 'border-emerald-600 bg-emerald-600'],
  ['Locked', 'border-amber-200 bg-amber-100'],
  ['Sold', 'border-gray-200 bg-gray-200'],
];

export default function Legend() {
  return (
    <ul className="flex flex-wrap justify-center gap-4 text-xs text-gray-600">
      {ITEMS.map(([label, look]) => (
        <li key={label} className="flex items-center gap-1.5">
          <span className={`h-4 w-4 rounded-t-md border ${look}`} aria-hidden="true" />
          {label}
        </li>
      ))}
    </ul>
```

```
  );
}
```

src/components/HoldTimer.jsx

```
import { useEffect, useState } from 'react';
import { formatTimeLeft } from '../utils/format';

// Ticks inside its own component so only this text re-renders every second, not the seat map
export default function HoldTimer({ expiresAt }) {
  const [now, setNow] = useState(Date.now);

  useEffect(() => {
    const timer = setInterval(() => setNow(Date.now()), 1000);
    return () => clearInterval(timer);
  }, []);

  const left = expiresAt - now; // derived from the deadline, so a late tick never drifts
  return (
    <p className={`text-sm ${left < 30000 ? 'font-semibold text-red-600' : 'text-gray-600'}`}>
      Seats held for <span role="timer" className="font-mono">{formatTimeLeft(left)}</span>
    </p>
  );
}
```

src/components/BookingSummary.jsx

```
import HoldTimer from './HoldTimer';
import { formatPrice } from '../utils/format';

const btn = 'rounded-md px-4 py-2 text-sm font-medium disabled:cursor-not-allowed disabled:opacity-50';
const primary = `${btn} bg-rose-600 text-white hover:bg-rose-700`;
const secondary = `${btn} border border-gray-300 text-gray-700 hover:bg-gray-50`;

export default function BookingSummary({ seats, total, maxSeats, phase, hold, bookingId, onHold, onPay,
onCancel, onNew }) {
  if (phase === 'booked') {
    return (
      <section className="rounded-lg border border-emerald-200 bg-emerald-50 p-4 text-center">
        <h2 className="font-semibold text-emerald-800">Booking confirmed</h2>
        <p className="mt-1 text-sm text-emerald-900">
          {bookingId} · Seats {seats.map((s) => s.id).join(', ')} · {formatPrice(total)}
        </p>
        <button type="button" className={`${secondary} mt-3 bg-white`} onClick={onNew}>
          Book more seats
        </button>
```

```
      </section>
    );
  }

  const isHeld = phase === 'held' || phase === 'paying';

  return (
    <section className="flex flex-wrap items-center justify-between gap-3 rounded-lg border border-
gray-200 p-4">
      <div>
        <p className="text-sm text-gray-700">
          {seats.length === 0
            ? `Pick up to ${maxSeats} seats`
            : `${seats.length} seat${seats.length > 1 ? 's' : ''}: ${seats.map((s) => s.id).join(',
')}`}
        </p>
        {isHeld && <HoldTimer key={hold.id} expiresAt={hold.expiresAt} />}
      </div>

      <div className="flex items-center gap-2">
        {isHeld && (
          <button type="button" className={secondary} onClick={onCancel} disabled={phase === 'paying'}>
            Change seats
          </button>
        )}
        {isHeld ? (
          <button type="button" className={primary} onClick={onPay} disabled={phase === 'paying'}>
            {phase === 'paying' ? 'Paying…' : `Pay ${formatPrice(total)}`}
          </button>
        ) : (
          <button type="button" className={primary} onClick={onHold} disabled={seats.length === 0 ||
phase === 'holding'}>
            {phase === 'holding' ? 'Holding seats…' : `Proceed ${formatPrice(total)}`}
          </button>
        )}
      </div>
    </section>
  );
}
```

src/App.jsx

```
import SeatMap from './components/SeatMap';
import Legend from './components/Legend';
import BookingSummary from './components/BookingSummary';
import { useSeatBooking } from './hooks/useSeatBooking';
import { getSelectedSeats, getTotal } from './utils/booking';
```

```
export default function App() {
  const { state, load, toggleSeat, hold, pay, cancel, newBooking } = useSeatBooking('show-42');

  if (state.status === 'loading') {
    return <p className="p-8 text-center text-gray-500" role="status">Loading seats…</p>;
  }
  if (state.status === 'error') {
    return (
      <div className="p-8 text-center" role="alert">
        <p className="text-red-600">Could not load seats: {state.error}</p>
        <button type="button" onClick={load} className="mt-3 rounded-md border px-4 py-2 text-sm">
          Retry
        </button>
      </div>
    );
  }

  const { show, maxSeats } = state.layout;

  return (
    <main className="mx-auto max-w-3xl space-y-5 p-4">
      <header>
        <h1 className="text-xl font-semibold">{show.title}</h1>
        <p className="text-sm text-gray-500">{show.venue} · {show.time}</p>
      </header>

      <Legend />

      <SeatMap
        layout={state.layout}
        seats={state.seats}
        selected={state.selected}
        locked={state.phase !== 'select'}
        onToggle={toggleSeat}
      />

      <p aria-live="polite" className="min-h-5 text-center text-sm text-amber-700">
        {state.notice}
      </p>

      <BookingSummary
        seats={getSelectedSeats(state)}
        total={getTotal(state)}
        maxSeats={maxSeats}
        phase={state.phase}
        hold={state.hold}
```

```
        bookingId={state.bookingId}
        onHold={hold}
        onPay={pay}
        onCancel={cancel}
        onNew={newBooking}
      />
    </main>
  );
}
```

How to see every path: pick F6 or G7 and wait 8 seconds before Proceed to get a 409. Hold seats and wait 2 minutes to see expiry. Hold, wait until under a second is left, then Pay to see the 410 path.

---

12. Wrap-up

Edge cases handled

- Sold / locked seats can't be selected; max 6 seats with a message.
- Concurrent booking: all-or-nothing hold on the server, 409 marks taken seats sold and keeps the rest.
- Hold expiry on the client (deadline-based timer) and on the server (TTL, 410 on Pay).
- Stale timers (holdId check), no local expiry during payment, no double Proceed / Pay.
- Stale load responses ignored on Retry / StrictMode.
- Keyboard and screen reader support; status never by colour alone.

Trade-offs to mention

Decision	Why	Cost
Server owns status, client shows "last known"	Only the server can resolve races	UI can be stale until the next hold attempt
Hold on Proceed, not on every seat click	Fewer requests, simpler	Conflicts show up later (at Proceed) instead of instantly
`useReducer` + phase enum	Impossible states impossible, testable without React	A bit more code than `useState` for the first version
Normalized `seats` map	O(1) updates and lookups	One extra transform on load
Plain buttons in flex rows	Accessible for free, easy	120 tab stops; roving tabindex would be nicer
Mock API in-process	No backend needed for the interview	Real network failures are only simulated

What to add with more time

- **Live updates:** WebSocket / SSE pushing `{ seatId, status }` events, or polling every 10 s, so seats grey out the moment someone else holds them.
- **Roving tabindex + arrow keys** (`role="grid"`), so the grid is one Tab stop and arrows move between seats.

- **"No single empty seat" rule** (see Q13 below).
- **"How many seats?" flow** that auto-selects N adjacent seats from the clicked one.
- **Release on leave:** `navigator.sendBeacon` on `pagehide` to free the hold early.
- **Idempotency key** on `confirmBooking` so a retried payment can't book twice.
- **Tests:** reducer unit tests (as in my self-check), React Testing Library for the flow, a mocked 409 / 410.
- **Big layouts:** render rows lazily or use canvas/SVG for stadium-size maps with zoom and pan.

---

13. Counter questions the interviewer may ask

REQUIREMENTS

Q1. Why do you need a hold step at all? Why not book directly on click?

Payment takes time (UPI, OTP, 3-D Secure). Without a hold, two users can both pay for the same seat and one has to be refunded. The hold reserves the seat for a short window so only one payment can succeed. It also makes the "Proceed" step a natural place to detect conflicts.

Q2. How would you support "How many seats?" asked up front, like BookMyShow?

Store `seatCount` in state. On click, select up to `seatCount` contiguous available seats in that row starting from the clicked one (skipping across an aisle is a product decision). If fewer fit, select what fits and let the user add the rest. Clicking again replaces the selection instead of adding.

Q3. How does the same component work for a flight?

The layout is data: rows, tiers (economy, extra legroom, business), `aisleAfter: [3]` for a 3-3 cabin or `[2, 6]` for 2-4-2, and missing seats for exits or galleys. I'd add `type: 'gap'` entries for missing seats. The reducer and API flow don't change.

STATE DESIGN

Q4. Why is "selected" not one of the seat statuses?

Seat status is server truth; selection is the user's local intent. If I stored `status: 'selected'` on the seat, any refresh from the server would wipe the selection, and I'd risk sending "selected" back as if it were a real status. Keeping them apart keeps the server data a faithful copy.

Q5. Why `useReducer` and not a few `useState`s?

The booking flow has five phases and most actions change several fields together (409 changes `seats`, `selected`, `phase` and `notice` at once). A reducer makes each transition one atomic update, guards actions by phase, and is a plain function I can test with asserts without rendering React.

Q6. Why normalize seats into a map instead of keeping rows with nested seats?

Updating one seat in nested arrays means mapping every row and every seat. With `seats[id]` it's one spread, lookups for price and labels are O(1), and unchanged seat objects keep their reference so `memo` skips them.

Q7. Should selection survive a page refresh?

Usually no for selection alone, but yes for an active hold: store `{ holdId, expiresAt }` in `sessionStorage` and on load ask the server if it is still valid. Otherwise the user loses a hold they still own and has to wait for it to expire.

CHALLENGE: CONCURRENCY

Q8. What if two users click "Proceed" for the same seat in the same millisecond?

The server must make "check and mark held" atomic: a DB transaction with a row lock / unique constraint, or Redis `SET seat:F6 holdId NX PX 120000` for each seat (with rollback, or a Lua script for all seats). Exactly one request wins; the other gets 409. The frontend can't solve this, it can only handle the 409 well.

Q9. Why all-or-nothing? Why not hold the free seats and report the taken ones?

A group of four wants to sit together. Holding two of them silently gives a booking the user didn't want, and those seats stay locked for others for minutes. All-or-nothing lets the user decide again with the full picture.

Q10. How would you show other users' bookings live?

Open a WebSocket (or SSE) per show. The server pushes `{ seatId, status }`; the reducer gets a `seatUpdate` action that changes `seats[id].status`, and if that seat is in `selected` and we are in `select` phase, drops it with a notice. Polling every 10 to 15 s is the cheap fallback. Either way, the hold call still validates, because a push can arrive late.

CHALLENGE: HOLD TIMER

Q11. Why not just do `setInterval` and decrement a seconds counter?

Browsers throttle timers in background tabs (to once a minute or less), and each tick can be late. A counter would drift and show time that the server no longer gives. Storing a deadline and computing `expiresAt - Date.now()` on each render is always correct, however late the tick.

Q12. What if the user's clock is wrong?

That's why the server sends `expiresInMs` and the client builds its own deadline from its own clock. Skew then doesn't matter, only network latency, which makes the client expire slightly early, which is the safe direction. The server's TTL is still the real rule.

BUSINESS RULES

Q13. How would you stop users leaving a single empty seat between booked ones?

Run a check on Proceed (not on every click, or the UI fights the user mid-selection). For each row, build the status after the proposed booking and look for a lone free seat between two taken seats or a taken seat and the row edge / aisle:

```
function leavesSingleGap(rowIds, taken) {
  return rowIds.some((id, i) =>
```

```
    !taken.has(id) &&
    (i === 0 || taken.has(rowIds[i - 1])) &&
    (i === rowIds.length - 1 || taken.has(rowIds[i + 1])));
}
```

Split the row at aisles before checking. Show "Please don't leave a single seat empty" and block Proceed.

IMPLEMENTATION

Q14. Why did you make each seat a `<button>` instead of a `<div onClick>`?

A button is focusable, works with Enter and Space, announces its label and pressed state, and supports `disabled` for sold seats, all for free. A div needs `role`, `tabIndex`, key handlers and `aria-*` added by hand, and people forget some of them.

Q15. 120 seats means 120 Tab stops. How would you fix that?

Roving tabindex: wrap the map in `role="grid"`, rows in `role="row"`, seats in `role="gridcell"` with a button. Only the active seat has `tabIndex=0`, others `-1`. Arrow keys move the active seat (skipping gaps), Home / End jump in a row, Space toggles. Tab then enters and leaves the map in one step.

Q16. Does the `memo` on `Seat` actually do anything here?

Yes, because every prop is stable: `seat` objects keep their reference unless that seat changed (normalized map + spread), `isSelected` and `locked` are booleans, `price` is a number, and `onToggle` comes from `useCallback` over `dispatch`. If I passed an inline `() => toggle(id)` it would break memo for every seat. I'd verify with the React DevTools profiler: one click should re-render one seat plus the summary.

PERFORMANCE / SCALE

Q17. What if the venue is a 50,000-seat stadium?

Don't render 50k DOM buttons. Show a stadium overview with blocks (sections) first; the user zooms into one block of a few hundred seats. For a single huge map, draw seats on a canvas or one SVG, do hit-testing on click, and keep an accessible list view (row / seat pickers) beside it. Live updates must be per block, not per stadium.

Q18. What if the API is slow?

Show a skeleton for the grid on load. For hold and pay, show the in-flight label and lock the UI (already done) and add a timeout with an `AbortController` so the user isn't stuck. For pay, never retry blindly: use an idempotency key so a retried request can't charge or book twice.

TRADE-OFFS

Q19. Why not optimistic UI for holding seats?

Optimistic UI is good when the server almost always agrees (a like button). Seat holds are exactly where the server often disagrees on popular shows. Showing "held" and then taking it back feels worse than a 400 ms

"Holding seats…". I use optimistic UI only for local selection, which can't fail.

Q20. Should "Change seats" wait for `releaseHold` to finish?

No. The user wants to go back right away, and if the release call fails the server TTL frees the seats anyway. The one risk: the user re-holds the same seats before the release lands, and the server answers 409 for their own seats. Fix it by sending the old `holdId` with the new hold request (`holdSeats(showId, seatIds, { replaces: oldHoldId })`) so the server releases and re-holds in one atomic step.

Q21. How would you test this?

Reducer unit tests for every transition (select, max, 409, stale expire, expire during pay, 410). Component tests with React Testing Library and a mocked API: click seats, Proceed, assert the countdown and Pay button; mock a 409 and assert the seat becomes "sold". Use fake timers (`vi.useFakeTimers`) to test expiry without waiting 2 minutes.